

1) Scenario Name

Role-Based Access Control

2) Weak Prompt

"Write test cases for user roles."

3) What I Feel is Missing

User roles are not defined (Admin, Manager, Customer, Guest, etc.).

No application/module specified.

No permissions or access rules provided.

No output format mentioned.

No positive/negative scenarios specified.

4) Rewritten Prompt

Context: Act as a Senior Test Engineer testing a Role-Based Access Control (RBAC) module in a CRM Web Application.

Instruction:

[MANDATORY] Generate exactly 10 test cases for Role-Based Access Control (RBAC).

[MANDATORY] Include Functional and Security test scenarios.

[MANDATORY] Include Positive and Negative test scenarios.

[Strict] Cover:

Admin Role Permissions

Manager Role Permissions

Viewer Role Permissions

Unauthorized Access Validation

Direct URL Access Restriction

Menu and Feature Visibility

Create/Edit/Delete Permission Validation

API Authorization Validation

Privilege Escalation Prevention

Audit Log Verification

Role Matrix:

Role	Permissions
------	-------------

Admin	Create, Edit, Delete, View Users and Reports
-------	--

Manager	Create, Edit, View Users and Reports
---------	--------------------------------------

Viewer	View Users and Reports only
--------	-----------------------------

Example:

TCID	Description	Priority	Preconditions	Steps	Expected Result
------	-------------	----------	---------------	-------	-----------------

TC001	Verify Admin can create a new user	High	Admin user logged in	Navigate to User Management and create a user	User created successfully
-------	------------------------------------	------	----------------------	---	---------------------------

Persona: Act as a Senior QA Engineer, Test Lead, and Security Test Analyst.

Output Format: Provide output in Excel-ready table format.

Tone: Use a professional and technical tone.

5) Observations

- 1: the second output is more complete because it covers all required user functionalities, while the first output only provides a general checklist.
- 2: The second output is easier to use for testing since it includes Test Case ID, Priority, Preconditions, Steps, and Expected Results in a structured format.
- 3: The second output is more professional and QA-focused, with a good mix of positive, negative, functional, and non-functional test scenarios.

6) Week prompt output:

When testing ****User Roles and Permissions**** (often referred to as RBAC - Role-Based Access Control), your main goal is to ensure that users can perform everything their role requires, and ***absolutely nothing*** they aren't authorized to do.

Here is a comprehensive template of test cases categorized by functional areas, designed to ensure robust security and usability.

1. Role Creation & Assignment (Admin Level)

These test cases verify that roles can be properly configured and assigned by authorized administrators.

Test Case ID	Scenario Description	Expected Result
---	---	---
TC_UR_001	Admin creates a new custom user role with specific permissions.	Role is created successfully and appears in the roles list.
TC_UR_002	Admin assigns a valid role to a new or existing user.	User's profile updates immediately; user inherits all role permissions.
TC_UR_003	Admin removes a role or downgrades a user's role.	User loses access to the restricted features immediately upon session refresh.
TC_UR_004	Admin attempts to delete a role that is currently assigned to active users.	System blocks deletion or prompts admin to reassign users first.

2. Positive Testing (Authorized Access)

Verify that users can seamlessly perform the actions explicit to their assigned roles.

* **TC_UR_005: Dashboard and UI Elements**

* *Action:* Log in as a specific role (e.g., *Editor*).

* *Expected:* The UI dynamically updates to show only relevant menus, buttons, and dashboards. No broken layout or hidden but clickable elements.

* **TC_UR_006: CRUD Operations (Create, Read, Update, Delete)**

* **Action:** Perform allowed actions based on the role matrix (e.g., *Manager* editing a report).

* **Expected:** Action completes successfully, data is saved to the database, and success messages display.

* **TC_UR_007: Multiple Roles (If supported)**

* **Action:** Assign multiple roles to a single user (e.g., *Billing Clerk* + *Support Agent*).

* **Expected:** User inherits the cumulative permissions of both roles without any conflicts or system crashes.

3. Negative Testing (Unauthorized Access)

These are critical security test cases to ensure users cannot escalate their privileges or bypass UI restrictions.

* **TC_UR_008: Direct URL Access (Force Browsing)**

* **Action:** Log in as a low-privilege user (e.g., *Viewer*), then paste a known Admin URL (e.g., `app.com/admin/settings`) directly into the browser.

* **Expected:** Access denied. System redirects to a `403 Forbidden` page or the home dashboard.

* **TC_UR_009: UI Element Bypass**

* **Action:** Inspect the webpage DOM (using Browser Developer Tools) to unhide or enable a button reserved for Admins (e.g., "Delete User"), then click it.

* *Expected:* System rejects the action on the backend/API level and returns an authorization error.

* **TC_UR_010: API Manipulation**

* *Action:* Intercept an API request using a tool like Postman or Burp Suite. Attempt to modify a record using a low-privilege user's authentication token.

* *Expected:* Backend rejects the request with an HTTP `401 Unauthorized` or `403 Forbidden` status code.

4. Edge Cases & Session Management

* **TC_UR_011: Real-time Permission Changes**

* *Action:* While User A is actively logged in and navigating the app, an Admin changes User A's role from *Admin* to *Viewer* in another session.

* *Expected:* On User A's next action/click, the system challenges the session, restricts access, or forces a re-login to apply new permissions.

* **TC_UR_012: Data Isolation Between Users**

* *Action:* Log in as User B who has the *same role* as User C (e.g., both are *Team Leads* but manage different teams). Try to access User C's team data via a direct object reference ID.

* *Expected:* Access denied. Users should only see data belonging to their specific scope, even if they share a general role.

****TC_UR_013: Session Expiry and Deactivation****

***Action:** Completely deactivate or lock a user account while they are logged in.

***Expected:** The user is immediately logged out on their next action and redirected to the login screen.

> **💡 Quick Testing Tip:** Always use a **Role-Permission Matrix** (a spreadsheet mapping roles on one axis and permissions on the other) before you write these test cases. It acts as your single source of truth and prevents you from missing overlapping permissions!